Windows runbook: from the copied folder to a verified model

Version 1.0 · 09 September 2026 · companion to the U-MV technical hand-over guide

This runbook is for one situation: a Windows 11 workstation with an NVIDIA GPU and Docker Desktop, the archive already copied to `D:` as `D:\Vegetiation\3_Mapping Acacia tortilis Trees\A.tortilis_Data_Model`, and Windows PowerShell as the shell. Do the steps in order. Every command is one line, even where it wraps on the page; copy it whole and paste one command at a time. Each step ends with what you should see.

Before you start

Check	How	Must be true
Docker Desktop installed and running	whale icon in the tray is steady; `docker --version` prints a version	yes
NVIDIA driver	`nvidia-smi` prints a table with the GPU name and a driver version ≥ 520	yes
Data folder	`D:\Vegetiation\3_Mapping Acacia tortilis Trees\A.tortilis_Data_Model` contains `Data used to build the model` and `A.tortilis Models`	yes
Free disk	≥ 40 GB on `C:` (Docker image)	yes
Internet	needed for the build and the first model download	yes
RAM	Task Manager → Performance → Memory shows the total	note the value

Open PowerShell (Start → type `PowerShell` → Enter). All commands below run there unless the step says "inside the container".

Step 1 — Give WSL2 (and therefore Docker) enough memory

The image build compiles software and failed once with `cannot allocate memory`. Set the limit once, below the machine's total RAM:

```
notepad $env:USERPROFILE\.wslconfig
```

Notepad opens (empty if the file did not exist). Make its content exactly (use 24GB on a 32 GB machine, 40GB on 64 GB), save with Ctrl+S, close:

```
[wsl2]
memory=24GB
swap=8GB
```

Then apply it:

```
wsl --shutdown
```

Wait until Docker Desktop's whale icon is steady again (it restarts its engine; about a minute).

You should see: no error from `wsl --shutdown`; Docker Desktop back to "Engine running".

Step 2 — Confirm Docker can use the GPU

```
docker run --rm --gpus all nvidia/cuda:11.8.0-base-ubuntu22.04 nvidia-smi
```

You should see: the same GPU table as `nvidia-smi`, printed from inside a container. Note the GPU name; it is needed in step 6.

If not: Docker Desktop → Settings → General: "Use the WSL 2 based engine" must be on; Settings → Resources → WSL integration: enable the default distribution; Apply & restart.

Step 3 — Go to the code folder and update it

The repository was already cloned inside the data folder. Update it (this also keeps the downloaded checkpoints):

```
cd "D:\Vegetiation\3_Mapping Acacia tortilis Trees\A.tortilis_Data_Model\U-MV-Acacia-tortilis-Crown-Mapping"
git pull
git log --oneline -1
```

You should see: `Already up to date.` or a list of updated files, then one line with a commit message about "build arguments" or later.

If the folder does not exist (fresh machine):

```
cd "D:\Vegetiation\3_Mapping Acacia tortilis Trees\A.tortilis_Data_Model"
git clone https://github.com/brakuta/U-MV-Acacia-tortilis-Crown-Mapping.git
cd U-MV-Acacia-tortilis-Crown-Mapping
```

Step 4 — Tell the container where the data and the weights are

```
copy docker\.env.example docker\.env
notepad docker\.env
```

Replace everything in Notepad with the three lines below (keep the quotes), save with Ctrl+S, close:

```
DATA_DIR="D:\Vegetiation\3_Mapping Acacia tortilis Trees\A.tortilis_Data_Model\Data used to build the model"
WEIGHTS_DIR="D:\Vegetiation\3_Mapping Acacia tortilis Trees\A.tortilis_Data_Model\A.tortilis Models\Pretrained w
eights"
HF_HUB_OFFLINE=0
```

Check it:

```
type docker\.env
```

You should see: the three lines exactly as above.

Step 5 — Check the data folder from Windows

```
$root = "D:\Vegetiation\3_Mapping Acacia tortilis Trees\A.tortilis_Data_Model\Data used to build the model"; for
each ($s in 'train','val','test2','Generalizability') { "{0,-18} img={1,6} ann={2,6}" -f $s, (Get-ChildItem "$ro
ot\img_dir\$s" -Filter *.tif -File).Count, (Get-ChildItem "$root\ann_dir\$s" -Filter *.tif -File).Count }
```

You should see:

```
train              img=  4893 ann=  4893
val                img=  2407 ann=  2407
test2              img=  3123 ann=  3123
Generalizability   img=  2162 ann=  2162
```

Step 6 — Find the GPU architecture number

From the GPU name of step 2:

GPU	CUDA_ARCH
TITAN RTX, RTX 2060 / 2070 / 2080, Quadro RTX	7.5
RTX A4000 / A5000 / A6000, RTX 3060 / 3070 / 3080 / 3090	8.6
A100	8.0
RTX 4070 / 4080 / 4090, RTX 6000 Ada	8.9

Step 7 — Build the image (30 to 60 minutes)

Use `CUDA_ARCH` from step 6; keep `MAX_JOBS=2` unless the machine has 64 GB RAM (then 4):

```
docker compose --env-file docker/.env -f docker/docker-compose.yml build --build-arg MAX_JOBS=2 --build-arg CUDA
_ARCH="7.5"
```

Leave the window open and the computer awake. The stages, in order: downloading the base image (`sha256:` lines), `apt` packages, `conda` packages, the MMCV compilation (long, few visible lines), `mamba-ssm`, the project install.

You should see: the last lines mention `umv 1.1.0` and the build ends without `ERROR`. Then:

```
docker images umv
```

prints one line with `umv` and `latest`.

If it ends with `ERROR` *and* `cannot allocate memory`*:* raise `memory=` in step 1, run `wsl --shutdown`, and repeat step 7; finished stages are reused. *Any other* `ERROR`*:* copy the last 60 lines and send them.

Step 8 — Start the container

```
docker compose --env-file docker/.env -f docker/docker-compose.yml run --rm umv
```

You should see: the prompt changes to `root@...:/workspace/U-MV#`. You are now inside Linux; the remaining steps run here. Check the mounts:

```
ls /data
ls /weights
nvidia-smi
```

You should see: `ann_dir img_dir`; the three `mambavision-*` folders; the GPU table.

If `/data` *is empty:* the path in step 4 is wrong; type `exit`, fix `docker\.env`, repeat step 8.

Step 9 — Verify the software (inside the container)

```
python tools/verify_install.py --variant small
```

The first run downloads the MambaVision-S encoder (~200 MB) from the Hugging Face Hub.

You should see: every library with a version, `CUDA available: True` with the GPU name, `mamba_ssm selective_scan_fn: ok`, `mmcv.ops ... ok`, a line `forward (1, 3, 512, 512) -> (1, 2, 512, 512)`, and `RESULT: OK`.

Step 10 — Verify the dataset (inside the container)

```
python tools/check_dataset.py /data --splits train val test2 Generalizability
```

You should see: the four pair counts of step 5 and `RESULT: OK`.

Step 11 — Identify the checkpoint (inside the container)

```
python tools/inspect_checkpoint.py "/weights/mambavision-s_generic-unet_acacia-88"
```

You should see: `"path": ".../best_mIoU_iter_95000.pth"`, `"variant": "small"`, `"iteration": 95000`.

Step 12 — Reproduce the published accuracy (inside the container)

```
CKPT="/weights/mambavision-s_generic-unet_acacia-88"
python tools/test.py configs/mambavision/U-MV-small.py "$CKPT" --test-split test2
python tools/test.py configs/mambavision/U-MV-small.py "$CKPT" --test-split Generalizability
```

Each run takes 10 to 20 minutes and ends with a metric table.

You should see: on `test2`, `mIoU` about 85.4 and `mFscore` about 91.6; on `Generalizability`, about 89.5 and 94.2. Copy both tables and send them. Optional: repeat with `U-MV-tiny.py` + `mambavision-t_generic-unet_acacia` and `U-MV-base.py` + `mambavision-b_generic-unet_acacia`.

Step 13 — Map one orthomosaic (inside the container, optional)

Put a GeoTIFF orthomosaic in a folder `orthos` inside `Data used to build the model` (from Windows), then:

```
python tools/geospatial_inference.py --config configs/mambavision/U-MV-small.py --checkpoint "$CKPT" --input "/data/orthos/<name>.tif" --output "/data/predictions/<name>_crowns.gpkg" --scratch-dir /tmp/geospatial_work --min-area 1.0 --save-prob
```

You should see: progress bars, then a JSON summary with `polygons`. The GeoPackage appears in Windows under `...\Data used to build the model\predictions` and opens in ArcGIS Pro or QGIS on top of the orthomosaic.

Step 14 — Leave and come back

`exit` leaves the container. Next time, only steps 3 (optional `git pull`) and 8 onward are needed; the image stays built.

If something else goes wrong

Send: the step number, the exact command, and the last 60 lines of output. The full reference is the technical hand-over guide (chapters 3 to 8) and its troubleshooting table.